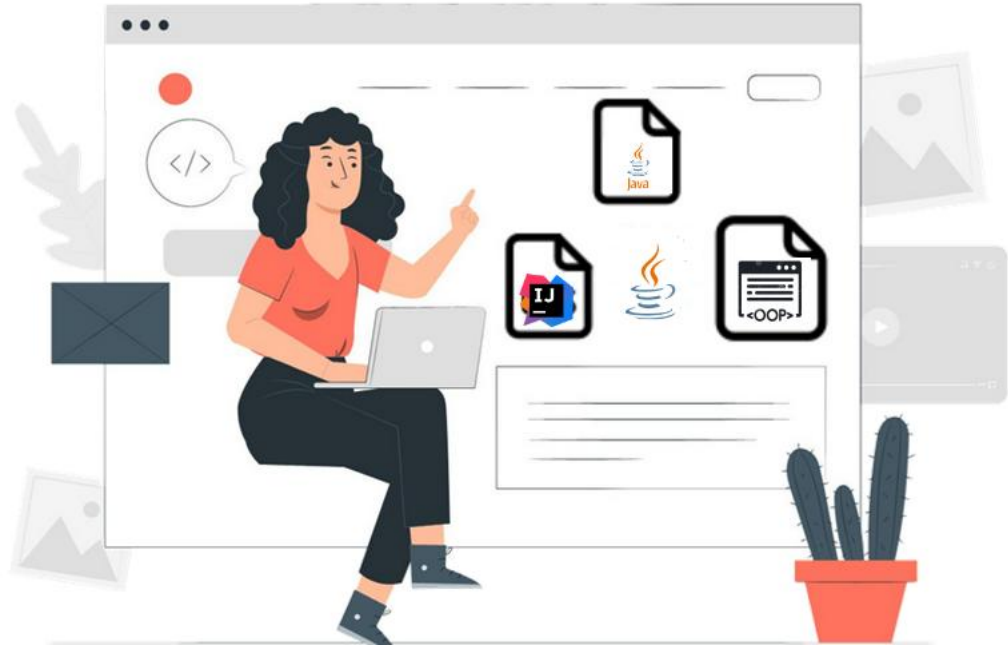
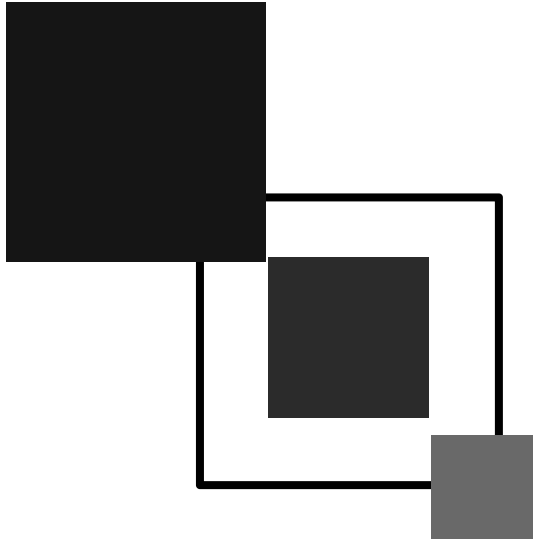




Domain Driven Design

Eliane Marion

FIAP

AGENDA DE TRABALHO



*Conexão com
Banco de Dados*



Inserir e Alterar



01

Métodos Inserir e Alterar

Classe EnderecoDao

Agora vamos fazer os métodos alterar e excluir, ambos possuem o mesmo conceito do método inserir.

Classe EnderecoDao - método alterar

```
public void alterar(Endereco endereco) {  
    conexao = GerenciadorBD.obterConexao();  
    PreparedStatement comandoSql = null;  
    try{  
        comandoSql = conexao.prepareStatement( sql: "update endereco set cep = ?, rua = ?, complemento = ?, "  
            "bairro = ?, cidade = ?, uf = ? where idEndereco = ?");  
        comandoSql.setString( parameterIndex: 1, endereco.getCep());  
        comandoSql.setString( parameterIndex: 2, endereco.getLogradouro()); //rua  
        comandoSql.setString( parameterIndex: 3, endereco.getComplemento());  
        comandoSql.setString( parameterIndex: 4, endereco.getBairro());  
        comandoSql.setString( parameterIndex: 5, endereco.getLocalidade()); //cidade  
        comandoSql.setString( parameterIndex: 6, endereco.getUf());  
        comandoSql.setInt( parameterIndex: 7, endereco.getId());  
        comandoSql.executeUpdate();  
        conexao.close();  
        comandoSql.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

ENTENDENDO O CÓDIGO

Criamos um método chamado `alterar` que recebe um `endereco` (objeto `endereco`) e não possui retorno.

```
public void alterar(Endereco endereco) {  
  
}
```

Obtemos a conexão através do método `obterConexao` que escrevemos na classe `GerenciadorBD`.

```
conexao = GerenciadorBD.obterConexao();
```

Nesta linha estamos criando um objeto chamado comandoSQL do tipo PreparedStatement. Dentro do try tentaremos alterar um registro no banco de dados.

```
PreparedStatement comandoSQL = null;
```

E a grande alteração que teremos é a sintaxe SQL...

```
comandoSQL = conexao.prepareStatement("update endereco set cep = ?,  
rua = ?, complemento = ?, bairro = ?, cidade = ?, uf = ? where  
idEndereco = ?");
```

- Nesta linha fazemos o uso da conexão com o banco de dados para preparar um statement com a instrução SQL update. Devemos escrever a instrução da mesma forma que escrevemos no banco de dados e no lugar dos valores colocaremos interrogação(?).

- Para cada interrogação utilizada devemos atribuir valores aos parâmetros indicados no preparedStatement. Devemos seguir a sequência utilizada na linha anterior.

```
comandoSQL.setString(1, endereco.getCep());  
comandoSQL.setString(2, endereco.getLogradouro());  
comandoSQL.setString(3, endereco.getComplemento());  
comandoSQL.setString(4, endereco.getBairro());  
comandoSQL.setString(5, endereco.getCidade());  
comandoSQL.setString(6, endereco.getUf());  
comandoSQL.setInt(7, endereco.getId());
```

- A única alteração foi a ordem, pois o id é o último valor já que entra na cláusula Where.

- Após montarmos nosso objeto comandoSQL o próximo passo é de fato executar a instrução update no banco, isso é feito com o método `executeUpdate()`.

```
comandoSQL.executeUpdate();
```

- Por fim fechamos nossa conexão

```
conexao.close();  
comandoSQL.close();
```



02

Testando a alteração no banco

Testando sua conexão

Podemos criar outra classe ou utilizarmos a classe TestaInsercao que já criamos.

```
public class TestaInsercao {  
    public static void main(String[] args) {  
        Scanner ent = new Scanner(System.in);  
        Endereco endereco = new Endereco();  
        EnderecoDao dao = new EnderecoDao();  
        System.out.println("Digite o código do endereco: ");  
        endereco.setId(ent.nextInt());  
        System.out.println("Digite o cep do endereco: ");  
        endereco.setCep(ent.nextLine());  
        System.out.println("Digite o nome da rua: ");  
        endereco.setLogradouro(ent.nextLine());  
    }  
}
```

Testando sua conexão

Continuação:

```
System.out.println("Digite o complemento do endereco: ");
endereco.setComplemento(ent.nextLine());
System.out.println("Digite o bairro do endereco: ");
endereco.setBairro(ent.nextLine());
System.out.println("Digite a cidade:");
endereco.setLocalidade(ent.nextLine());
System.out.println("Digite o estado:");
endereco.setUf(ent.nextLine());
/*Esta próxima linha chama o método alterar que criamos na classe
EnderecoDao, lembre-se que o id deve existir na tabela.*/
dao.update(endereco);
System.out.println("Endereço atualizado com sucesso!");
```



03

Excluindo um endereço



Método excluir - EnderecoDao

FIA/P

```
public void excluir(int id) {  
    conexao = GerenciadorBD.obterConexao();  
    PreparedStatement comandoSql = null;  
    try{  
        comandoSql = conexao.prepareStatement( sql: "delete from endereco where idEndereco = ?");  
        comandoSql.setInt( parameterIndex: 1,id);  
        comandoSql.executeUpdate();  
        conexao.close();  
        comandoSql.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

ENTENDENDO O CÓDIGO

Criamos um método chamado `excluir` que recebe um endereço (objeto endereço) e não possui retorno, poderíamos alterar a assinatura deste método recebendo um inteiro: `public void excluir (int id).`

```
public void excluir(int id) {  
  
}
```

Obtemos a conexão através do método `obterConexao` que escrevemos na classe `GerenciadorBD`.

```
conexao = GerenciadorBD.obterConexao();
```

Nesta linha estamos criando um objeto chamado comandoSQL do tipo PreparedStatement. Dentro do try tentaremos alterar um registro no banco de dados.

```
PreparedStatement comandoSQL = null;
```

E a grande alteração que teremos é a sintaxe SQL...

```
comandoSQL = conexao.prepareStatement("delete from endereco where  
idEndereco = ?");
```

- Nesta linha fazemos o uso da conexão com o banco de dados para preparar um statement com a instrução SQL delete. Devemos escrever a instrução da mesma forma que escrevemos no banco de dados e no lugar dos valores colocaremos interrogação(?).

- Neste caso só temos o id, portanto teremos uma única linha atribuindo valor ao parâmetro indicado no PreparedStatement.

```
comandoSQL.setInt(1, id);
```

- Não esqueça que o id deve existir no banco e devemos colocar a cláusula Where, caso contrário excluimos todos registros da tabela.

- Após montarmos nosso objeto comandoSQL o próximo passo é de fato executar a instrução update no banco, isso é feito com o método `executeUpdate()`.

```
comandoSQL.executeUpdate();
```

- Por fim fechamos nossa conexão

```
conexao.close();  
comandoSQL.close();
```



04

Testando a exclusão no banco

Testando o método excluir

Podemos criar outra classe ou utilizarmos a classe TestaInsercao que já criamos.

```
public class TestaInsercao {  
    public static void main(String[] args) {  
        Scanner ent = new Scanner(System.in);  
        Endereco endereco = new Endereco();  
        EnderecoDao dao = new EnderecoDao();  
        System.out.println("Digite o código do endereco: ");  
        int id = ent.nextInt();  
        dao.remover(id);  
        System.out.println("Endereco excluido!");  
    }  
}
```

Entendendo seu código

Mas esta forma não é a mais adequada, já que o usuário está digitando um número (id) e estamos chamando o método excluir passando esse número e "mandando" excluir no banco, porém não temos certeza se este código existe no banco de dados.

- Então o ideal seria passar um endereço para o método excluir, mas antes fazer uma pesquisa no banco para verificar se existe esse endereço no banco.

Método excluir passando o endereço como parâmetro

```
public void excluir(Endereco endereco) {  
    conexao = GerenciadorBD.obterConexao();  
    PreparedStatement comandoSql = null;  
    try{  
        comandoSql = conexao.prepareStatement( sql: "delete from endereco where idEndereco = ?");  
        comandoSql.setInt( parameterIndex: 1, endereco.getId());  
        comandoSql.executeUpdate();  
        conexao.close();  
        comandoSql.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

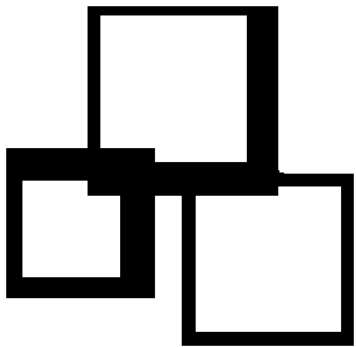
Testando o método excluir

```
public class TestaInsercao {  
  
    public static void main(String[] args) {  
        Scanner ent = new Scanner(System.in);  
        Endereco endereco = new Endereco();  
        EnderecoDao dao = new EnderecoDao();  
        System.out.println("Digite o id do endereco: ");  
        int id = ent.nextInt();  
        endereco = dao.buscarPorId(id);  
        dao.remover(endereco);  
        System.out.println("Endereco excluido!");  
    }  
}
```

Entendendo seu código

Agora temos um problema, porque para excluirmos um registro no banco precisamos antes consulta-lo, por isso precisamos construir um método de busca, que deverá fazer um select no banco e guardar os dados no objeto.

```
endereco = dao.buscarPorId(id);
```



OBRIGADO

To be continued...

BYE
BYE